

Method Overloading in Java

If a class has multiple methods having same name but different in parameters, it is known as Method Overloading.

If we have to perform only one operation, having same name of the methods increases the readability of the program.

Suppose you have to perform addition of the given numbers but there can be any number of arguments, if you write the method such as `a(int,int)` for two parameters, and `b(int,int,int)` for three parameters then it may be difficult for you as well as other programmers to understand the behavior of the method because its name differs.

So, we perform method overloading to figure out the program quickly.

Advantage of method overloading

Method overloading increases the readability of the program.

Different ways to overload the method

There are two ways to overload the method in java

1. By changing number of arguments
2. By changing the data type

In Java, Method Overloading is not possible by changing the return type of the method only.

1) Method Overloading: changing no. of arguments

In this example, we have created two methods, first `add()` method performs addition of two numbers and second `add` method performs addition of three numbers.

In this example, we are creating static methods so that we don't need to create instance for calling methods.

```
class Adder
{
    static int add(int a,int b)
    {
        return a+b;
    }
    static int add(int a,int b,int c)
    {
        return a+b+c;
    }
}
```

```

    }
}
class TestOverloading1
{
    public static void main(String[] args)
    {
        System.out.println(Adder.add(11,11));
        System.out.println(Adder.add(11,11,11));
    }
}

```

2) Method Overloading: changing data type of arguments

In this example, we have created two methods that differs in data type. The first add method receives two integer arguments and second add method receives two double arguments.

```

class Adder
{
    static int add(int a, int b)
    {
        return a+b;
    }
    static double add(double a, double b)
    {
        return a+b;
    }
}
class TestOverloading2
{
    public static void main(String[] args)
    {
        System.out.println(Adder.add(11,11));
        System.out.println(Adder.add(12.3,12.6));
    }
}

```

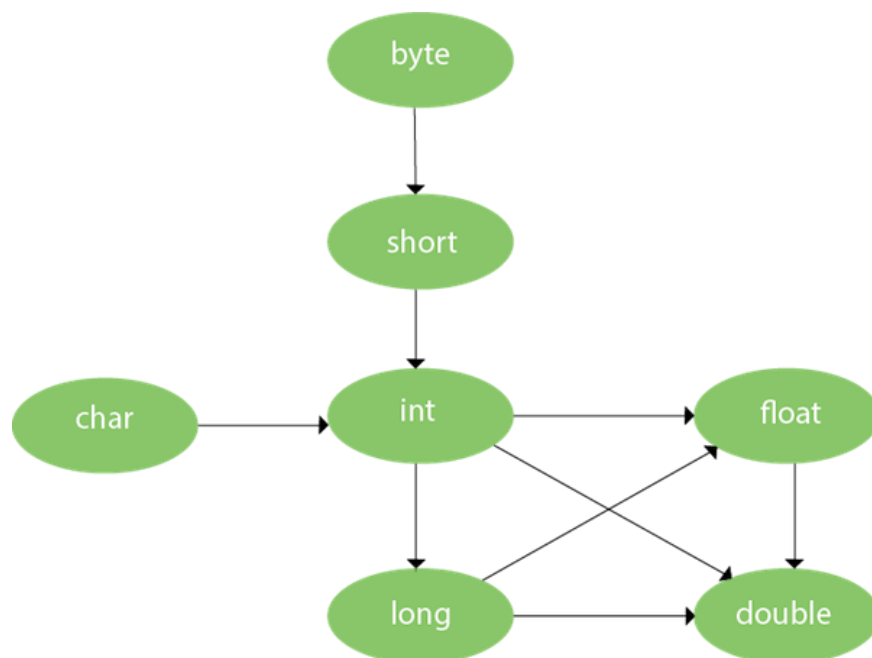
Can we overload java main() method?

Yes, by method overloading. You can have any number of main methods in a class by method overloading. But JVM calls main() method which receives string array as arguments only. Let's see the simple example:

```
class TestOverloading4
{
    public static void main(String[] args)
    {
        System.out.println("main with String[]");
    }
    public static void main(String args)
    {
        System.out.println("main with String");
    }
    public static void main()
    {
        System.out.println("main without args");
    }
}
```

Method Overloading and Type Promotion

One type is promoted to another implicitly if no matching datatype is found. Let's understand the concept by the figure given below:



As displayed in the above diagram, byte can be promoted to short, int, long, float or double. The short datatype can be promoted to int, long, float or double. The char datatype can be promoted to int, long, float or double and so on.

Example of Method Overloading with Type Promotion

```
class OverloadingCalculation1
```

```
{
    void sum(int a,long b)
    {
        System.out.println(a+b);
    }
    void sum(int a,int b,int c)
    {
        System.out.println(a+b+c);
    }

    public static void main(String args[])
    {
        OverloadingCalculation1 obj=new OverloadingCalculation1();
        obj.sum(20,20); //now second int literal will be promoted to long
        obj.sum(20,20,20);
    }
}
```

Example of Method Overloading with Type Promotion if matching found

If there are matching type arguments in the method, type promotion is not performed.

```
class OverloadingCalculation2
```

```
{
    void sum(int a,int b)
    {
        System.out.println("int arg method invoked");
    }
    void sum(long a,long b)
    {
        System.out.println("long arg method invoked");
    }
    public static void main(String args[])
    {
        OverloadingCalculation2 obj=new OverloadingCalculation2();
        obj.sum(20,20); //now int arg sum() method gets invoked
    }
}
```

Example of Method Overloading with Type Promotion in case of ambiguity

If there are no matching type arguments in the method, and each method promotes similar number of arguments, there will be ambiguity.

```
class OverloadingCalculation3
{
    void sum(int a,long b)
    {
        System.out.println("a method invoked");
    }
    void sum(long a,int b)
    {
        System.out.println("b method invoked");
    }

    public static void main(String args[])
    {
        OverloadingCalculation3 obj=new OverloadingCalculation3();
        obj.sum(20,20);//now ambiguity
    }
}
```

